COMP 3306 Operating Systems: Midterm Review

Chapters 1 to 6

Exam format: 70 minutes, 100 points. Multiple choice, true/false, fill in the blank, short answer, and problems (Gantt charts, diagrams, small calculations).

Chapter 1: Introduction

- **What an OS is:** the program between the user and the hardware; the kernel is always running.
- **Interrupts and traps:** a trap (exception) is a software-generated interrupt from an error or a system call.
- **DMA:** the device moves a whole block to memory with one interrupt per block, not per byte.
- **Dual mode:** user mode vs. kernel mode, the mode bit, and privileged instructions.
- **Timer:** why the OS sets a timer before running a user program.
- Multiprogramming vs. multitasking (time sharing).
- **Program vs. process:** a program is passive, a process is a program in execution.

Chapter 2: Operating-System Structures

- **OS services:** user interface, program execution, I/O, file system, communication, error detection.
- **System calls:** how a program asks the OS for a service; API example: printf() calls write() system call.
- **Types of system calls:** process control, file, device, information, communication, protection.
- **Policy vs. mechanism:** policy is "what will be done", mechanism is "how to do it".
- **OS structures:** monolithic, layered, microkernel (message passing, slower), modules, hybrid.

Chapter 3: Processes

- **Memory layout of a process:** text, data (initialized and uninitialized), heap, stack. Need to know where variables go.
- Five process states (new, ready, running, waiting, terminated) and the arrows between them.
- **PCB (Process Control Block):** what it stores (state, program counter, registers, etc.).
- **Context switch:** save old state, load new state; why it is overhead.
- fork(), exec(), wait(), exit(); zombie vs. orphan process.
- **IPC:** shared memory vs. message passing.

Chapter 4: Threads

- **Thread vs. process:** threads share code, data, and files, but each has its own stack, registers, and program counter.
- **Benefits of threads:** responsiveness, resource sharing, economy, scalability.
- Concurrency vs. parallelism; data vs. task parallelism.
- **Multithreading models:** many-to-one, one-to-one, many-to-many.
- Single-threaded vs. multithreaded server calculation (the class example). Practice with different numbers.

Chapter 5: CPU Scheduling

- **Scheduling criteria:** CPU utilization, throughput, turnaround time, waiting time, response time.
- Preemptive vs. nonpreemptive scheduling; the dispatcher.
- FCFS (convoy effect), SJF (minimum average waiting time), Round Robin (time quantum), priority (starvation and aging).
- Be able to draw a Gantt chart and compute waiting time, turnaround time, and averages.
- Waiting time = turnaround time − burst time; turnaround time = completion time − arrival time.

Chapter 6: Synchronization Tools

- **Race condition:** why producer(counter++) and consumer (counter−−) running together can give a wrong value.
- **Critical section and its three requirements:** mutual exclusion, progress, bounded waiting.
- **Mutex lock:** acquire() and release(); busy waiting (spinlock).
- **Semaphores:** wait() and signal(); counting vs. binary semaphore. Be able to trace the value of S step by step (like the parking lot example).
- **Monitors (basic idea) and liveness:** deadlock and starvation.